

DOA Hand- In 1

September 7, 2026

1 Exercise 1(a)

For this exercise, we chose arbitrary numbers for M and N , and created a vector A using M . For the random integers, we chose that the range should be $[0, 999]$, then counted how many of the N random integers in the range that were in A using an iterator.

The code:

```
[1]: #include <iostream>
#include <vector>
#include <cstdlib>
#include <ctime>

using namespace std;

void Exercise1() {
    // Ensure different random sequences on each run
    srand(time(nullptr));

    // Our selections for M and N
    int M = 1000;
    int N = 500;

    // Generate M random integers between 0 and 999, and put them in vector A
    vector<int> A;
    for (int i = 0; i < M; i++) {
        A.push_back(rand() % 1000);
    }

    // Generate N random integers between 0 and 999, and count how many of them
    // are in A using an iterator
    int count = 0;
    for (int j = 0; j < N; j++) {
        int target = rand() % 1000;

        // Search through A iteratively
        // Update count if integer is found at least once in A, then break loop
        for (vector<int>::const_iterator it = A.begin(); it != A.end(); ++it) {
```

```

        if (*it == target) {
            count++;
            break;
        }
    }

    cout << "Number of N random integers found in A: " << count << endl;
}

// Run code
Exercise1();

```

Number of N random integers found in A: 309

This generates an output of the number of times that the N random integers appeared in A once or more.

2 Exercise 1(b)

The vector initialization takes $O(M)$ exactly.

The outer for-loop (line 24) repeats N times, with a worst case of searching through the vector M times, meaning $O(M \times N)$.

The rest of the lines takes $O(1)$, as they each run exactly once.

This means that overall we get

$$O(1) + O(M) + O(M \times N) = O(M \times N)$$

3 Exercise 2

The nested for-loops take $O(N \times N \times (N \times \sqrt{N})) = O(N^{3.5})$.

The next for-loop takes $O(N \times N) = O(N^2)$ meaning that overall we get

$$O(N^{3.5}) + O(N^2) = O(N^{3.5})$$

since $N^{3.5}$ is the dominating term.

4 Exercise 3(a)

Analyses of running times:

4.1 (1)

This takes $O(n)$.

4.2 (2)

This takes $O(n^2)$.

4.3 (3)

This takes $O(n \times (n \times n)) = O(n^3)$.

4.4 (4)

The inner for-loop repeats $\sum_{i=0}^{n-1} i$ times. This is just a standard **Gauss sum**, which can be written as $\frac{n^2-n}{2}$, meaning that the algorithm takes $O(\frac{n^2-n}{2}) = O(\frac{1}{2}n^2 - \frac{1}{2}n) = O(n^2)$.

5 Exercise 3(b) & 3(c)

Implementations with test cases in C++.

Our code:

```
[ ]: #include <iostream>
#include <chrono> // For running time measurements

using namespace std;
using namespace std::chrono;

void fragment1(long long n) {
    long long sum = 0;
    for (long long i = 0; i < n; ++i) ++sum;
}

void fragment2(long long n) {
    long long sum = 0;
    for (long long i = 0; i < n; ++i)
        for (long long j = 0; j < n; ++j) ++sum;
}

void fragment3(long long n) {
    long long sum = 0;
    for (long long i = 0; i < n; ++i)
        for (long long j = 0; j < n * n; ++j) ++sum;
}

void fragment4(long long n) {
    long long sum = 0;
    for (long long i = 0; i < n; ++i)
        for (long long j = 0; j < i; ++j) ++sum;
}

// Calculate running-time for given code fragment and return milliseconds
```

```

double runningTime(const char* name, void (*fragment)(long long), long long n) {

    // Start timer
    auto t1 = high_resolution_clock::now();

    // Run code fragment
    fragment(n);

    // End timer
    auto t2 = high_resolution_clock::now();

    double elapsed_time = duration<double, milli>(t2 - t1).count();

    // Print running-time
    // Print given N
    cout << name << " (N = " << n << "): " << elapsed_time << " ms\n";

    return elapsed_time;
}

void run() {
    // Run test cases for each code fragment
    double f1_t0 = runningTime("Fragment 1", fragment1, 500000000);
    double f1_t1 = runningTime("Fragment 1", fragment1, 1000000000);
    cout << "\n";

    double f2_t0 = runningTime("Fragment 2", fragment2, 20000);
    double f2_t1 = runningTime("Fragment 2", fragment2, 40000);
    cout << "\n";

    double f3_t0 = runningTime("Fragment 3", fragment3, 750);
    double f3_t1 = runningTime("Fragment 3", fragment3, 1500);
    cout << "\n";

    double f4_t0 = runningTime("Fragment 4", fragment4, 30000);
    double f4_t1 = runningTime("Fragment 4", fragment4, 60000);
    cout << "\n";

    // Print growth rates of fragments
    cout << "Growth rates T(N_1)/T(N_0) \n";
    cout << "Fragment 1 growth rate: " << (f1_t1 / f1_t0) << "\n";
    cout << "Fragment 2 growth rate: " << (f2_t1 / f2_t0) << "\n";
    cout << "Fragment 3 growth rate: " << (f3_t1 / f3_t0) << "\n";
    cout << "Fragment 4 growth rate: " << (f4_t1 / f4_t0) << "\n";
}

run();

```

Fragment 1 (N = 500000000): 320.826 ms
 Fragment 1 (N = 1000000000): 632.512 ms

Fragment 2 (N = 20000): 285.035 ms
 Fragment 2 (N = 40000): 980.411 ms

Fragment 3 (N = 750): 393.094 ms
 Fragment 3 (N = 1500): 3370.35 ms

Fragment 4 (N = 30000): 223.218 ms
 Fragment 4 (N = 60000): 1069.62 ms

Growth rates $T(N_1)/T(N_0)$

Fragment 1 growth rate: 1.97151
 Fragment 2 growth rate: 3.43962
 Fragment 3 growth rate: 8.57391
 Fragment 4 growth rate: 4.7918

As we can see, even if algorithms share Big-O notations, their running times can differ by a lot.

	Expected growth rate	Actual growth rate
Fragment 1	≈ 2	1.97
Fragment 2	$\approx 2^2 = 4$	3.44
Fragment 3	$\approx 2^3 = 8$	8.57
Fragment 4	$\approx 2^2 = 4$	4.79

Though they aren't exact, they do however still follow the pattern

$$O(N) < O(N^2) < O(N^3)$$

which is to be expected.

6 Exercise 3(d)

The running times from earlier were done using a kernel in VSCode. To be sure that they were correct, we ran the program again with and without optimizers in a separate terminal and got the following results:

Without any optimizers:

```
g++ -O0 H1Ex3.cpp -o no_opt
./no_opt
```

Fragment 1 (N = 500000000): 406.856 ms
 Fragment 1 (N = 1000000000): 742.401 ms

Fragment 2 (N = 20000): 297.459 ms
 Fragment 2 (N = 40000): 1230.26 ms

```
Fragment 3 (N = 750): 322.297 ms
Fragment 3 (N = 1500): 2643.89 ms
```

```
Fragment 4 (N = 30000): 336.182 ms
Fragment 4 (N = 60000): 1391.04 ms
```

Growth rates $T(N_1)/T(N_0)$

```
Fragment 1 growth rate: 1.82472
Fragment 2 growth rate: 4.13591
Fragment 3 growth rate: 8.20329
Fragment 4 growth rate: 4.13776
```

With optimizer flag `-O`:

```
g++ -O H1Ex3.cpp -o opt
./opt
```

```
Fragment 1 (N = 500000000): 0 ms
Fragment 1 (N = 1000000000): 4.2e-05 ms
```

```
Fragment 2 (N = 20000): 4.2e-05 ms
Fragment 2 (N = 40000): 0 ms
```

```
Fragment 3 (N = 750): 0 ms
Fragment 3 (N = 1500): 4.2e-05 ms
```

```
Fragment 4 (N = 30000): 0 ms
Fragment 4 (N = 60000): 4.1e-05 ms
```

Growth rates $T(N_1)/T(N_0)$

```
Fragment 1 growth rate: inf
Fragment 2 growth rate: 0
Fragment 3 growth rate: inf
Fragment 4 growth rate: inf
```

As we can see, the run-time with the optimizer is instantaneous, so it would be much preferred.

7 Exercise 4(a)

Explanation is spread as comments in the code.

Our implementation:

```
[ ]: #include <vector>
      #include <stdexcept>
      #include <cassert>

      using namespace std;
```

```

class MaxHeap
{
public:
    virtual bool isEmpty() const = 0;
    virtual int size() = 0;
    virtual void insert(const int x) = 0;
    virtual const int findMax() const = 0;
    virtual int deleteMax() = 0;
};

// Our implementation
class VectorMaxHeap : public MaxHeap {
private:
    vector<int> data; // Vector containing class data

public:
    // Check if heap is empty
    bool isEmpty() const override {
        return data.empty();
    }

    // Returns number of elements
    int size() override {
        return data.size();
    }

    // Adds element to heap
    void insert(const int x) override {
        data.push_back(x);
    }

    // Finds the maximal value in the heap through iteration
    const int findMax() const override {

        assert(!isEmpty()); // Heap must not be empty

        vector<int>::const_iterator it = data.begin(); // Initialize iteration

        // Set first element of the heap as max value and continue iteration
        int maxVal = *it;
        ++it;

        // Iterate through the entire heap, and update max value accordingly
        while (it != data.end()) {
            if (*it > maxVal) {
                maxVal = *it;
            }
        }
    }
};

```

```

        }
        ++it;
    }

    return maxVal;
}

// Find, delete and return max value in heap
int deleteMax() override {

    assert(!isEmpty()); // Heap must not be empty

    vector<int>::iterator it = data.begin(); // Initialize iterator
    vector<int>::iterator tempMax = it; // Saves address of current max value

    // Set first element of the heap as max value and continue iteration
    int maxVal = *it;
    ++it;

    // Iterate through the entire heap, and update max value accordingly
    // Save address of the maximal value for deletion later
    while (it != data.end()) {
        if (*it > maxVal) {
            maxVal = *it;
            tempMax = it;
        }
        ++it;
    }

    // Delete max value from heap, then return it
    data.erase(tempMax);
    return maxVal;
}
};

```

In our solution `VectorMaxHeap` implements the virtual functions of `MaxHeap` as intended.

8 Exercise 4(b)

`isEmpty()` has constant time complexity since it only checks for a single element. If it is empty, it returns true, if it is not, it returns false, so $O(1)$.

`size()` also takes $O(1)$ since the size can be given by a single arithmetic operation such as `end - begin`.

`insert()` has a worst case of $O(N)$, if it were required to reallocate every single element before inserting.

`findMax()` the while-loop's worst case scenario would be that the data elements are ordered from

lowest to highest, meaning that the maximal value would need to be updated through every iteration $(N-1)$ comparisons and $(N-1)$ maximum value updates), so the function takes $O((N-1) + (N-1)) = O(2N-2) = O(N)$.

`deleteMax()` iterates $N-1$ times for maximum value comparisons, and the worst case would be that the elements were ordered from lowest to highest, meaning $N-1$ maximum value updates as well as the address saver for the maximum value. This iteration in itself takes $O(N)$ for the same reasons as `findMax()`. Deletion of the maximum value using `erase()` has its own time complexity, as the worst case here would be that the first element would be the largest, thereby meaning that every following element would have to be shifted, possibly taking $O(N-1) = O(N)$.

The total time complexity of the function is therefore $O(N) + O(N) = O(N)$.

9 Exercise 5(a)

Explanation is spread as comments in the code.

Our implementation:

```
[ ]: #include <iostream>
#include <vector>
#include <cassert>

using namespace std;

class Book {
private:
    int id;
    int category;
    bool borrowed;

public:
    Book(int id, int category) : id(id), category(category), borrowed(false) {}

    int getId() const { return id; }
    int getCategory() const { return category; }
    bool isBorrowed() const { return borrowed; }

    void setBorrowed(bool status) { borrowed = status; }

    void display() const {
        cout << "Book ID: " << id
              << "\nCategory: " << category
              << "\nStatus: " << (borrowed ? "Borrowed" : "Available")
              << endl;
    }
};

class Library {
```

```

private:
    vector<Book> books;
    vector<int> categoryCount = vector<int>(16, 0); // Vector of size 16
    →initialized to 0
public:
    void addBook(int id, int category) {

        // Check category validity
        if (category < 0 || category > 15) {
            cout << "Invalid category\n";
            return;
        }

        // Go through books to check if ID matches book already
        for (const auto& b : books) {
            if (b.getId() == id) {
                cout << "Book is already in library\n";
                return;
            }
        }

        // Add book to vector and update category count
        books.push_back(Book(id, category));
        categoryCount[category]++;

        // Warning fragment if category exceeds given limit
        if (!books.empty()) {

            // Check number of categories with books in them
            int validCategories = 0;
            for (int i = 0; i < 16; i++) {
                if (categoryCount[i] > 0) {
                    validCategories++;
                }
            }

            if (validCategories > 0) {
                double currentAverage = static_cast<double>(books.size()) /
                →validCategories;
                // Issue warning if category exceeds limit
                if ((categoryCount[category] + 1) > 2.0 * currentAverage) {
                    cout << "Warning: Adding Book exceeds category limit" <<
                →endl;
                }
            }
        }
    }
}

```

```

bool borrowBook(int id) {
    // Go through books to check if it is there
    for (auto& b : books) {
        if (b.getId() == id) {
            // If book is found, check if it is borrowed
            if (!b.isBorrowed()) {
                // Book is not borrowed
                // Update to book being borrowed and return true
                b.setBorrowed(true);
                return true;
            }
            // Book was found but is already borrowed
            return false;
        }
    }
    // Book was not found in the library
    return false;
}

bool returnBook(int id) {
    // Go through books to check if it is there
    for (auto& b : books) {
        if (b.getId() == id) {
            // If book is found and borrowed update status to not borrowed,
            ↪and return true
            if (b.isBorrowed()) {
                b.setBorrowed(false);
                return true;
            }
            // Book was found but is not set as borrowed
            return false;
        }
    }
    // Book was not found in the library
    return false;
}

void displayAllBooks() const {
    // Go through all books and display them one by one
    for (const auto& b : books) {
        b.display();
    }
}

void displayAvailableBooks() const {
    // Go through all books and display the ones that aren't borrowed

```

```

        for (const auto& b : books) {
            if (!b.isBorrowed()) {
                b.display();
            }
        }
    }

void displayBorrowedBooks() const {
    // Go through all books and display the ones that are borrowed
    for (const auto& b : books) {
        if (b.isBorrowed()) {
            b.display();
        }
    }
}

int countBooksInCategory(int category) const {
    // Check if category is valid (0-15)
    if (category >= 0 && category <= 15) {
        return categoryCount[category]; // Return number of books in the
        category
    }
    return 0;
}

};

void Main() {
    Library myLibrary;

    // Adding books to the library
    myLibrary.addBook(1, 3);
    myLibrary.addBook(2, 7);
    myLibrary.addBook(3, 3);
    myLibrary.addBook(4, 15);
    myLibrary.addBook(5, 3);
    myLibrary.addBook(6, 0);

    // Display all books in the library
    cout << "\nAll books in the library:\n";
    myLibrary.displayAllBooks();

    // Display available books
    cout << "\nAvailable books in the library:\n";
    myLibrary.displayAvailableBooks();

    // Borrow a book by ID
    cout << "\nBorrowing Book ID 2:\n";

```

```

    if (myLibrary.borrowBook(2)) {
        cout << "Book ID 2 borrowed successfully.\n";
    } else {
        cout << "Failed to borrow Book ID 2.\n";
    }

    // Attempt to borrow the same book again
    cout << "\nAttempting to borrow Book ID 2 again:\n";
    if (myLibrary.borrowBook(2)) {
        cout << "Book ID 2 borrowed successfully.\n";
    } else {
        cout << "Failed to borrow Book ID 2.\n";
    }

    // Display borrowed books
    cout << "\nBorrowed books in the library:\n";
    myLibrary.displayBorrowedBooks();

    // Return a borrowed book by ID
    cout << "\nReturning Book ID 2:\n";
    if (myLibrary.returnBook(2)) {
        cout << "Book ID 2 returned successfully\n";
    } else {
        cout << "Failed to return Book ID 2\n";
    }

    // Display available books after returning
    cout << "\nAvailable books in the library after returning Book ID 2:\n";
    myLibrary.displayAvailableBooks();

    // Display the number of books in a specific category
    int categoryToCheck = 3;
    cout << "\nNumber of books in category " << categoryToCheck << ": "
        << myLibrary.countBooksInCategory(categoryToCheck) << endl;
}

Main();

```

We ran the program and got the following output:

Warning: Adding Book exceeds category limit

All books in the library:

Book ID: 1

Category: 3

Status: Available

Book ID: 2

Category: 7

Status: Available
Book ID: 3
Category: 3
Status: Available
Book ID: 4
Category: 15
Status: Available
Book ID: 5
Category: 3
Status: Available
Book ID: 6
Category: 0
Status: Available

Available books in the library:

Book ID: 1
Category: 3
Status: Available
Book ID: 2
Category: 7
Status: Available
Book ID: 3
Category: 3
Status: Available
Book ID: 4
Category: 15
Status: Available
Book ID: 5
Category: 3
Status: Available
Book ID: 6
Category: 0
Status: Available

Borrowing Book ID 2:
Book ID 2 borrowed successfully.

Attempting to borrow Book ID 2 again:
Failed to borrow Book ID 2.

Borrowed books in the library:
Book ID: 2
Category: 7
Status: Borrowed

Returning Book ID 2:
Book ID 2 returned successfully

Available books in the library after returning Book ID 2:

Book ID: 1
Category: 3
Status: Available
Book ID: 2
Category: 7
Status: Available
Book ID: 3
Category: 3
Status: Available
Book ID: 4
Category: 15
Status: Available
Book ID: 5
Category: 3
Status: Available
Book ID: 6
Category: 0
Status: Available

Number of books in category 3: 3

The output suggests that our implementation works as intended.

10 Exercise 5(b)

We chose `vectors` to store the books in the library, as they provide flexibility and fits the objective well. The dynamic sizing of a vector means we didn't have to know the number of books prior to making our library, and searching through the entirety of a vector is generally easy (like using `auto` for example), which made functions where that is needed (such as the `display` functions) very manageable.

11 Exercise 5(c)

We'll go through each function inside the `Library` class.

`addBook(id, category)`: Time complexity $O(n)$.

This is because $O(n)$ is the dominating term, as the function requires searching through the entire vector of books to verify whether the book already exists before inserting. Inserting the book takes amortized $O(1)$, and evaluating the category balance warning requires looping over a fixed range of 16 categories, which takes constant time $O(16) = O(1)$.

`borrowBook(id)`: Time complexity $O(n)$.

This is because $O(n)$ is the dominating term, as the function requires searching through the entire vector of books in the worst case to locate the book with the matching ID. Updating the book's borrowed status once found takes constant time $O(1)$.

`returnBook(id)`: Time complexity $O(n)$.

$O(n)$ is the dominating term, as the function requires searching through the entire vector of books in the worst case to locate the matching ID before resetting its borrowed status in $O(1)$ time.

`displayAllBooks()`: Time complexity $O(n)$.

This is because the function requires iterating through the entire vector of books from beginning to end to print the details of every single book, performing an $O(1)$ print operation for each of the n books.

`displayAvailableBooks()`: Time complexity $O(n)$.

The function requires inspecting the entire collection of n books, evaluating each book's borrowed status, and printing the ones that are available.

`displayBorrowedBooks()`: Time complexity $O(n)$.

This function requires inspecting the entire collection of n books, evaluating each book's borrowed status, and printing those that are currently borrowed.

`countBooksInCategory(category)`: Time complexity $O(1)$.

This is because the category count is accessed directly using its index in the vector of books, which would take constant time.